

Cost-Aware Diffusion Draft Trees for Speculative Decoding

Shuai Zhang^{1,2} Huachuan Qiu² Hongliang He² Yong Dai²

¹Zhejiang University ²Westlake University
zhangshuai@westlake.edu.cn

Abstract

Speculative decoding accelerates inference by having a lightweight drafter propose tokens verified in parallel by the target language model. Block diffusion drafters such as DFlash generate an entire draft block in one pass, yielding per-position marginals; DDTree uses these to build a candidate tree that maximizes expected acceptance length under a fixed node budget. We observe, however, that acceptance length is non-decreasing in budget: it always favors larger trees regardless of verification cost, offering no principled basis for budget selection. We introduce **CaDDTree** (Cost-aware Diffusion Draft Tree), a method that directly optimizes token throughput (expected tokens generated per unit time) by jointly selecting the tree structure and node budget. We model draft and verification latencies explicitly, show that the throughput objective decomposes into a per-round one-dimensional search over the budget, and prove that under a convex verification cost the throughput function is *unimodal*, enabling an efficient greedy stopping rule. CaDDTree requires no offline budget search, adapting the budget each round from the current per-position distributions and verification cost. Experiments on Qwen3-4B and Qwen3-8B across eight benchmarks spanning reasoning, coding, and instruction-following tasks show that CaDDTree matches or surpasses DDTree with oracle budget selection on nearly all tasks.

1 Introduction

Autoregressive language models generate text token by token, requiring a full forward pass at every step. Speculative decoding (Leviathan et al., 2023; Chen et al., 2023) addresses this bottleneck by pairing a lightweight drafter with the target language model: the drafter proposes candidate tokens, and the target model verifies them in a single parallel forward pass, preserving the output distribution.

Tree-based verification (Miao et al., 2024) extends this by organizing multiple candidate contin-

uations as a tree: common prefixes are shared and all branches are verified in a single parallel forward pass. Each additional node raises coverage but also increases verification cost.

This tension between coverage and cost is the central optimization problem in draft tree construction: *how many nodes should the tree have, and where should they be placed?* Existing methods (Wang et al., 2025; Ringel and Romano, 2026; Li et al., 2024a; Liu et al., 2026a) optimize the draft tree to maximize expected token acceptance. While intuitive, this objective is misaligned with throughput: acceptance length is non-decreasing in budget and provides no principled basis for budget selection. Methods that adapt tree shape per step (Li et al., 2024a; Liu et al., 2026b) still fix the total node budget offline, leaving no mechanism to respond to per-round variability in the optimal budget.

We propose throughput as the primary design criterion for draft tree construction. By modeling drafting and verification costs as functions of tree size, we derive a per-round objective that decomposes into a one-dimensional budget search. The central finding is that under a mild *convexity condition* on the verification cost, the throughput objective is unimodal in tree size. This unimodality yields an efficient greedy algorithm that finds the throughput-optimal tree and adapts the budget per round, without any offline budget search or additional training.

We instantiate the framework on block diffusion drafting (Arriola et al., 2025; Chen et al., 2026), which produces per-position marginal distributions in a single forward pass. Experiments on Qwen3-4B and Qwen3-8B across eight benchmarks spanning reasoning, coding, and instruction-following tasks show that CaDDTree matches or surpasses DDTree with oracle budget selection on nearly all tasks. Code is available at <https://github.com/ZhangShuai1230/CaDDTree>.

2 Related Work

Speculative decoding and drafter design. Speculative decoding (Leviathan et al., 2023; Chen et al., 2023) uses a cheap drafter to propose tokens verified in one parallel forward pass by the target model, preserving its distribution. Autoregressive drafters conditioned on target hidden states, most notably the EAGLE family (Li et al., 2024b,a, 2026), achieve high acceptance rates. Block diffusion models (Arriola et al., 2025) offer a complementary paradigm: a single forward pass predicts an entire token block, enabling longer drafts at near-constant latency. DFlash (Chen et al., 2026) instantiates this by conditioning the drafter on target-model features, achieving state-of-the-art performance. PARD (An et al., 2025) adapts autoregressive models to mimic block-parallel prediction.

Tree-based parallel verification. SpecInfer (Miao et al., 2024) introduced tree attention, enabling verification of an entire candidate tree in one forward pass. Medusa (Cai et al., 2024) exploits this with lightweight prediction heads covering successive future positions. EAGLE-2 (Li et al., 2024a) constructs draft trees dynamically guided by per-step acceptance-rate estimates; EAGLE-3 (Li et al., 2026) further improves draft quality by fusing multi-layer features. DySpec (Xiong et al., 2025), Staged Speculative Decoding (Spector and Re, 2023), and Recursive Speculative Decoding (Jeon et al., 2024) explore further structural extensions. DART (Liu et al., 2026a) builds draft trees from one-pass parallel logits with n -gram continuity constraints. TALON (Liu et al., 2026b) adapts tree shape per step using token confidence; GOOSE (Jin et al., 2026) builds anisotropic trees routing reliable tokens into chains and uncertain ones into wide branches. Both fix the total node budget offline.

Optimal tree construction. OPT-Tree (Wang et al., 2025) first formulated adaptive tree construction for autoregressive drafters, maximizing expected acceptance length under a fixed node budget. DDTree (Ringel and Romano, 2026) extends this to block diffusion drafters: one pass yields per-position marginals for greedy optimal tree assembly. Zhang et al. (Zhang et al., 2026) optimize throughput directly via reinforcement learning but require training additional draft policies. CaDDTree departs from all of the above: it replaces acceptance-length maximization with direct

throughput optimization, adapts the total node budget per round, and provably identifies the optimum via a greedy algorithm with unimodality guarantees, requiring no additional training beyond a one-time cost profiling step.

3 Background

3.1 Speculative Decoding

Let $p(\cdot \mid \mathbf{c})$ denote the target model’s distribution over the next token given context $\mathbf{c}$, and let $q(\cdot \mid \mathbf{c})$ denote the drafter’s distribution. Speculative decoding (Leviathan et al., 2023; Chen et al., 2023) accelerates generation by having the drafter propose a sequence of n candidate tokens $(x_1, \dots, x_n)$, then letting the target model verify all of them in a single parallel forward pass. For each position i , the target evaluates $p(x_i \mid \mathbf{c}, x_{<i})$ and accepts or rejects via rejection sampling (Leviathan et al., 2023): accepted tokens extend the output, while the first rejection triggers a resample from a corrected distribution, preserving the target’s distribution exactly. Under greedy decoding, acceptance is equivalent to requiring $x_i = \arg \max p(\cdot \mid \mathbf{c}, x_{<i})$.

Each round begins with one committed token b , the *bonus token*: it comes from the prefill pass in the first round and from the last accepted token in subsequent rounds. Let $\alpha \geq 0$ denote the number of tokens accepted in a round; the round commits $1 + \alpha$ tokens in total. Throughput scales with $\mathbb{E}[\alpha]$, making draft quality the central design objective.

3.2 Block Diffusion Drafting

Autoregressive drafters require one forward pass per drafted token. Block diffusion models (Arriola et al., 2025) eliminate this cost: a single forward pass given context $\mathbf{c}$ and bonus token b produces logits for L positions simultaneously, yielding a per-position *marginal distribution*

$$q_i(v \mid \mathbf{c}, b) = \text{softmax}(\ell_i)_v, \quad (1)$$

for $v \in \mathcal{V}$ and $i = 1, \dots, L$. Each q_i is a *marginal*: it conditions only on $(\mathbf{c}, b)$, not on tokens chosen at earlier positions within the same block, so the drafter’s joint distribution factorizes as

$$Q(y_{1:L} \mid \mathbf{c}, b) := \prod_{i=1}^L q_i(y_i \mid \mathbf{c}, b). \quad (2)$$

This one-pass, fixed-cost structure makes long draft sequences feasible regardless of depth; the trade-off is that positions are predicted independently,

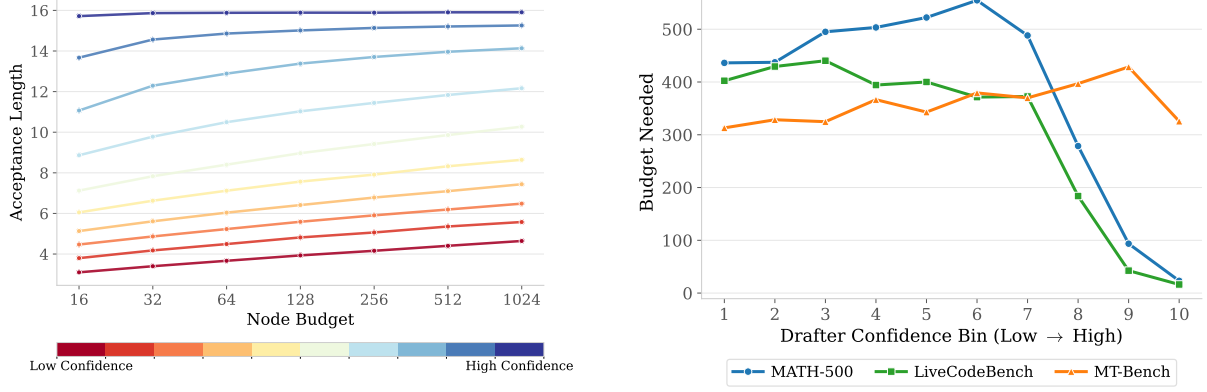


Figure 1: *Left*: Acceptance length vs. node budget for ten drafter-confidence deciles (Bin 1: low, Bin 10: high), Qwen3-4B on MATH-500 at temperature 0.0. High-confidence rounds saturate at small budgets; low-confidence rounds continue to benefit from larger trees throughout the range. *Right*: Budget required to recover 100% of the 1024-node acceptance length across confidence deciles for three datasets (Qwen3-4B, temperature 0.0). All decrease with confidence; reasoning tasks drop sharply while MT-Bench decreases later and more gradually.

which can accelerate error accumulation. DFlash (Chen et al., 2026) instantiates block diffusion conditioned on target-model features, achieving state-of-the-art acceptance rates.

3.3 Tree Drafting and Tree Attention

Rather than proposing a single continuation, the drafter can propose a *tree* of candidates verified simultaneously in one target-model forward pass via *tree attention* (Miao et al., 2024). Covering multiple branches increases the probability that the target’s preferred continuation appears somewhere in the draft.

Formally, a *draft tree* $\mathcal{T}$ is a set of nodes, where each node $\mathbf{s} = (s_1, \dots, s_d)$ represents the depth- d token sequence $(b, s_1, \dots, s_d)$; $\mathcal{T}$ is *valid* if it is prefix-closed: whenever $\mathbf{s} \in \mathcal{T}$, every prefix of $\mathbf{s}$ also belongs to $\mathcal{T}$. In tree attention, each node attends only to the shared context KV cache and to its own ancestors. The target model selects the path with the longest accepted prefix; that prefix is appended to the output, and the first unaccepted token along the chosen path becomes the next bonus token.

3.4 Draft Tree Construction

Given a draft tree $\mathcal{T}$, the ideal objective is to maximize the expected acceptance length under the target distribution p :

$$\max_{\mathcal{T}} \mathbb{E}_p[\alpha(\mathcal{T}, \mathbf{c})], \quad (3)$$

where $\alpha(\mathcal{T}, \mathbf{c})$ denotes the number of tokens accepted. This is intractable: p is the expensive

target model itself, and evaluating $\mathbb{E}_p[\alpha]$ requires marginalizing over all paths in $\mathcal{T}$ under p .

Ringel and Romano (2026) propose to replace p by the drafter’s factorized distribution Q from Equation (2), yielding a tractable surrogate. Under Q , the expected number of accepted tokens decomposes over nodes as

$$\Phi(\mathcal{T}) := \sum_{\mathbf{s} \in \mathcal{T}} \pi_{\mathbf{s}}, \quad (4)$$

where the *prefix probability*

$$\pi_{\mathbf{s}} := \prod_{i=1}^d q_i(s_i \mid \mathbf{c}, b) \quad (5)$$

is the probability that a sample from Q begins with $\mathbf{s}$. The factorization of Q is what makes $\Phi(\mathcal{T})$ computable from a single drafter forward pass.

Optimizing $\Phi(\mathcal{T})$ under a fixed budget $n = |\mathcal{T}|$ reduces to selecting the n nodes with the largest prefix probabilities. Labeling all prefix probabilities in non-increasing order as $\delta_1 \geq \delta_2 \geq \dots$,

$$\Phi^*(n) := \max_{|\mathcal{T}|=n} \Phi(\mathcal{T}) = \sum_{k=1}^n \delta_k, \quad (6)$$

with the optimal tree $\mathcal{T}_n^*$ consisting of the top- n nodes by $\pi_{\mathbf{s}}$. DDTree constructs $\mathcal{T}_n^*$ greedily: a max-heap over log-prefix-scores pops nodes in order of decreasing δ_k , pushing the first child and next sibling at each step, and halts after B pops. The tree size B is a hyperparameter with no principled criterion for setting it.

Table 1: Per-token generation time (ms/tok, lower is better) and mean acceptance length τ at temperature 0.0. n^* is the oracle’s best grid budget; $\bar{n}^*(\sigma)$ is CADDTREE’s per-round mean (std). CADDTREE matches or surpasses DDTree-oracle on nearly all tasks while selecting its budget automatically each round.

Dataset	AR	DFlash		DDTree-oracle			CADDTREE (ours)		
	ms/tok	ms/tok	τ	ms/tok	τ	n^*	ms/tok	τ	$\bar{n}^*(\sigma)$
<i>Qwen3-4B</i>									
MATH-500	30.65	5.93	7.71	4.53	10.35	256	4.53	10.37	221 (57)
GSM8K	31.11	6.96	6.53	5.14	9.08	256	5.18	9.13	249 (61)
AIME 2025	31.64	6.28	7.43	4.95	9.40	128	4.96	9.64	215 (37)
HumanEval	30.87	6.82	6.64	4.97	9.32	256	5.01	9.32	224 (59)
MBPP	30.68	7.41	6.14	5.26	8.87	256	5.31	8.91	257 (59)
LiveCodeBench	30.82	6.57	7.09	4.97	9.31	128	4.93	9.64	197 (56)
MT-Bench	31.07	12.59	4.39	8.34	6.47	256	8.32	6.44	235 (37)
Alpaca	31.08	16.34	3.12	10.42	4.89	256	10.66	4.88	264 (41)
<i>Qwen3-8B</i>									
MATH-500	30.95	5.83	7.91	4.58	10.20	128	4.60	10.32	165 (42)
GSM8K	30.87	6.99	6.51	5.22	8.81	128	5.19	9.04	184 (24)
AIME 2025	31.44	6.11	7.49	5.13	9.18	128	5.00	9.51	156 (40)
HumanEval	31.10	7.00	6.48	5.14	8.98	128	5.11	9.20	172 (38)
MBPP	31.01	7.75	5.93	5.53	8.42	128	5.53	8.61	187 (20)
LiveCodeBench	30.66	6.56	7.17	5.03	9.37	128	5.04	9.53	152 (49)
MT-Bench	30.38	12.71	4.28	8.59	6.10	128	8.58	6.22	172 (25)
Alpaca	30.14	15.78	3.08	10.43	4.68	128	10.34	4.80	186 (11)

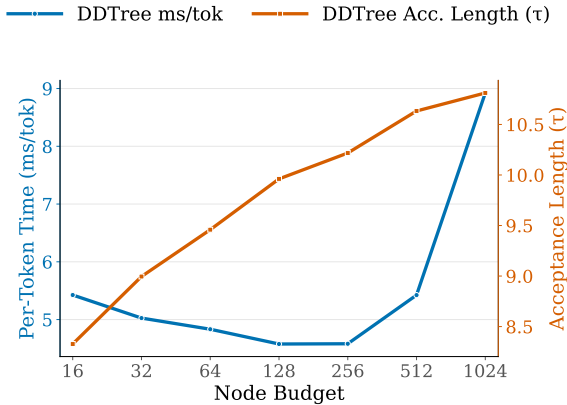


Figure 2: Acceptance length τ (right axis) and per-token generation time (left axis) vs. node budget, for Qwen3-4B on MATH-500 at temperature 0.0. Acceptance length is monotone; per-token time is unimodal and minimized at a budget well below the acceptance-length maximum.

4 Throughput-Optimal Draft Tree Construction

4.1 The Fixed-Budget Trap

The DDTree framework frames tree construction as: given a budget B , build $\mathcal{T}_B^*$. This leaves the budget as a free hyperparameter, and there is no principled criterion for setting it. Three empirical observations motivate replacing the acceptance-length objective with direct throughput maximization over an adaptive, context-aware budget.

Observation 1: per-token time is unimodal, not monotone. Figure 2 plots acceptance length and per-token generation time as functions of the node budget for Qwen3-4B on MATH-500. Acceptance length increases monotonically: more nodes give the target model a larger set of candidates to accept. Per-token time, however, is unimodal: it decreases initially because larger trees yield more accepted tokens per round, making the fixed drafting cost cheaper per output token, then increases once the marginal verification overhead outweighs the additional acceptance gain. The throughput-optimal budget is strictly smaller than the acceptance-length-maximizing budget: the latter always selects a larger tree than throughput can justify.

Observation 2: the optimal budget varies from round to round. The optimal budget is not a property of the dataset alone; it varies at each decoding round depending on the drafter’s confidence. Figure 1 quantifies this using Qwen3-4B on MATH-500, grouping rounds into ten confidence deciles by their maximum position-1 probability.

The left panel shows acceptance length as a function of budget per decile. High-confidence rounds saturate quickly (further nodes add negligible acceptance gain), while low-confidence rounds continue to benefit from larger trees throughout the budget range. The right panel shows that the budget needed for full acceptance recovery varies sub-

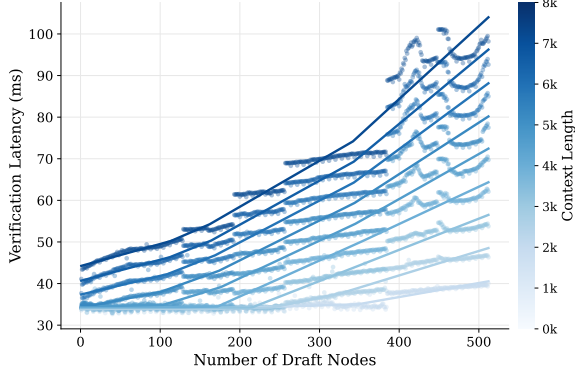


Figure 3: Verification latency vs. number of draft nodes for Qwen3-4B on 8×A800 across nine context lengths (0–8k tokens, light to dark). Dots are raw measurements; curves are convex regression fits. Latency is monotone and convex in node count at every context length; longer contexts shift the curve upward and increase its slope.

stantially with drafter confidence across all three datasets; reasoning tasks saturate earlier and more sharply than MT-Bench. No single fixed budget is efficient across all confidence levels and tasks.

Observation 3: verification cost grows with tree size and context. Figure 3 shows that verification latency increases with node count, and becomes more expensive at longer context lengths. This matters because context length grows throughout generation, so verification cost keeps changing even within a single response.

4.2 Throughput as the Optimization Objective

The observations above suggest that per-token latency, rather than acceptance length, should be the design criterion.

Cost model. Let ℓ denote the current context length. Define the round cost $C(\mathcal{T}; \ell) := C_d + C_v(|\mathcal{T}|; \ell)$, where $C_d > 0$ is the drafting cost and $C_v(n; \ell)$ is the verification cost for a tree with n nodes at context length ℓ . We model C_v as a function of n and ℓ ; Section 5.3 validates the resulting cost estimates.

Per-round objective. Let $\mathcal{T}_t$ denote the draft tree built at round t and α_t the number of tokens accepted. The response throughput over R rounds is $\Theta_R = \sum_t (1 + \alpha_t) / \sum_t C(\mathcal{T}_t; \ell_t)$. Under the surrogate approximation (derivation in Appendix A.1), maximizing long-run throughput reduces round-by-round to maximizing

$$\hat{\theta}(\mathcal{T}; \ell) := \frac{1 + \Phi(\mathcal{T})}{C_d + C_v(|\mathcal{T}|; \ell)}. \quad (7)$$

4.3 Decomposition into Budget Selection

The objective $\hat{\theta}(\mathcal{T}; \ell)$ ranges over all valid trees. We first reduce this combinatorial problem to a search over a single integer.

Proposition 1 (Structure-Budget Decomposition). *For any $n \geq 0$, the valid tree with exactly n nodes maximizing $\hat{\theta}(\cdot; \ell)$ is the greedy best-first tree $\mathcal{T}_n^*$. Consequently,*

$$\max_{\mathcal{T} \text{ valid}} \hat{\theta}(\mathcal{T}; \ell) = \max_{n \geq 0} \hat{\theta}^*(n; \ell), \quad (8)$$

$$\hat{\theta}^*(n; \ell) := \frac{1 + \Phi^*(n)}{C_d + C_v(n; \ell)}.$$

Proposition 1 reduces the problem (proof: Appendix A.2) to the one-dimensional search $\max_{n \geq 0} \hat{\theta}^*(n; \ell)$, evaluated incrementally via the same best-first heap used to build the DDTree, with $\delta_n = \Phi^*(n) - \Phi^*(n-1)$ being the n -th largest prefix probability.

4.4 Unimodality of Surrogate Throughput

All proofs in this section are in Appendix A. We identify a general condition under which $\max_n \hat{\theta}^*(n; \ell)$ is efficiently solvable. Define the *marginal verification cost* $c_n := C_v(n; \ell) - C_v(n-1; \ell)$ (for fixed ℓ) and the *marginal acceptance-cost ratio*

$$\rho_n := \frac{\delta_n}{c_n}. \quad (9)$$

Assumption 1 (Diminishing Benefit-Cost Ratio). *The marginal benefit-cost ratios are non-increasing: $\rho_1 \geq \rho_2 \geq \dots$.*

Lemma 2 (Weighted Average Representation). *For any $n \geq 0$ with $c_{n+1} > 0$,*

$$\hat{\theta}^*(n+1) = w_n \hat{\theta}^*(n) + (1 - w_n) \rho_{n+1}, \quad (10)$$

where $w_n = \frac{C_d + C_v(n; \ell)}{C_d + C_v(n+1; \ell)} \in (0, 1)$. Consequently,

$$\hat{\theta}^*(n+1) > \hat{\theta}^*(n) \iff \rho_{n+1} > \hat{\theta}^*(n). \quad (11)$$

Theorem 3 (Unimodality of Surrogate Throughput). *Under Assumption 1, $\{\hat{\theta}^*(n)\}_{n \geq 0}$ is unimodal: there exists $n^* \geq 0$ such that $\hat{\theta}^*(n)$ is non-decreasing for $n \leq n^*$ and non-increasing for $n \geq n^*$.*

Corollary 4 (Greedy Optimality). *The first index n^* satisfying $\hat{\theta}^*(n^*+1) \leq \hat{\theta}^*(n^*)$ is the global maximizer of $\hat{\theta}^*$, and $\mathcal{T}_{n^*}^*$ is the throughput-optimal tree.*

Corollary 5 (Convex cost). *If $C_v(\cdot; \ell)$ is convex in n for fixed ℓ (marginal cost c_n non-decreasing), Assumption 1 holds.*

Algorithm 1 CADDTREE: Cost-Aware Diffusion Draft Tree Construction

Require: Per-position distributions $\{q_i\}_{i=1}^L$ with top- K tokens $\{v_i^{(k)}\}$ and log-probs $\{\log q_i^{(k)}\}$; current context length ℓ ; drafting cost C_d ; verification cost function $C_v(\cdot; \ell)$

Ensure: Throughput-optimal draft tree $\mathcal{T}^*$

```

1:  $\mathcal{T}^* \leftarrow \emptyset, \Phi \leftarrow 0, n \leftarrow 0$ 
2:  $\hat{\theta}_{\text{prev}} \leftarrow 1/(C_d + C_v(0; \ell))$ 
3: Initialize max-heap  $H$  with  $((1), \log q_1^{(1)})$ 
4: while  $H \neq \emptyset$  do
5:   Pop  $\mathbf{s} = (s_1, \dots, s_d)$  with  $\sigma(\mathbf{s}) = \sum_{i=1}^d \log q_i^{(s_i)}$ 
6:    $\delta \leftarrow \exp(\sigma(\mathbf{s}))$ 
7:    $\hat{\theta}_{\text{new}} \leftarrow (1 + \Phi + \delta) / (C_d + C_v(n+1; \ell))$ 
8:   if  $\hat{\theta}_{\text{new}} \leq \hat{\theta}_{\text{prev}}$  then
9:     break
10:  end if
11:  Add  $(v_1^{(s_1)}, \dots, v_d^{(s_d)})$  to  $\mathcal{T}^*$ 
12:   $\Phi += \delta; n += 1; \hat{\theta}_{\text{prev}} \leftarrow \hat{\theta}_{\text{new}}$ 
13:  if  $s_d + 1 \leq K$  then
14:    Push sibling  $(s_1, \dots, s_{d-1}, s_d + 1)$ 
    with key  $\sigma(\mathbf{s}) - \log q_d^{(s_d)} + \log q_d^{(s_d+1)}$ 
15:  end if
16:  if  $d < L$  then
17:    Push first child  $(s_1, \dots, s_d, 1)$  with key
     $\sigma(\mathbf{s}) + \log q_{d+1}^{(1)}$ 
18:  end if
19: end while
20: return  $\mathcal{T}^*$ 

```

4.5 The CADDTREE Algorithm

Algorithm 1 implements Corollary 4 for any verification cost $C_v(\cdot; \ell)$ satisfying Assumption 1 at the current context length ℓ . It runs a max-heap over log-prefix-probabilities and halts at the first node that fails the stopping condition (11). The tree size n^* adapts jointly to the drafter’s confidence distribution and the context-dependent cost, without any hyperparameter.

4.6 Offline Cost Profiling

Profiling procedure. $C_v(n; \ell)$ is estimated once offline per (model, hardware) pair. We profile at a grid of context lengths $\mathcal{L} = \{0, 1k, 2k, \dots, 8k\}$ tokens and node counts $\mathcal{N} = \{1, 2, \dots, N\}$ with $N = 1024$. At each (ℓ, n) , we run the target model with a dummy tree of n nodes and record the median verification latency over 20 trials (after 5 warm-up iterations). The drafting cost C_d is

measured as the mean latency of a single drafter forward pass over 500 rounds after 5 warm-up iterations.

Convex regression. For each $L_j \in \mathcal{L}$, we fit $\hat{C}_v(\cdot; L_j)$ by solving

$$\min_{f \text{ convex, non-decreasing}} \sum_{n \in \mathcal{N}} (y_{j,n} - f(n))^2, \quad (12)$$

a convex quadratic program with $O(N)$ variables and $O(N)$ constraints, solved in under one second per context length via CLARABEL (Goulart and Chen, 2026). The result is a piecewise-linear function that is convex and non-decreasing by construction, with breakpoints automatically determined by the data.

Context interpolation at inference. At run time, the current context length ℓ is known exactly. For $L_j \leq \ell < L_{j+1}$, let $\alpha = (\ell - L_j)/(L_{j+1} - L_j)$; we set

$$\hat{C}_v(n; \ell) = (1 - \alpha) \hat{C}_v(n; L_j) + \alpha \hat{C}_v(n; L_{j+1}). \quad (13)$$

Since linear interpolation of convex functions is convex, $\hat{C}_v(\cdot; \ell)$ inherits convexity and monotonicity in n , so Assumption 1 holds for every ℓ .

5 Experiments

5.1 Experimental Setup

Models. We evaluate on two target models: Qwen3-4B and Qwen3-8B (Yang et al., 2025), each paired with its DFlash checkpoint.

Benchmarks. Eight benchmarks spanning reasoning, code, and instruction following: MATH-500 (Lightman et al., 2024), GSM8K (Cobbe et al., 2021), AIME 2025, HumanEval (Chen et al., 2021), MBPP (Austin et al., 2021), LiveCodeBench (Jain et al., 2025), MT-Bench (Zheng et al., 2023), and Alpaca (Taori et al., 2023).

Baselines.

- **AR:** standard greedy/sampling without speculative decoding.
- **DFlash** (Chen et al., 2026): block diffusion speculative decoding with a single sequence draft.
- **DDTree-oracle** (Ringel and Romano, 2026): DDTree with the best budget per (dataset, model, temperature), selected by grid search over $\{16, 32, 64, 128, 256, 512, 1024\}$.

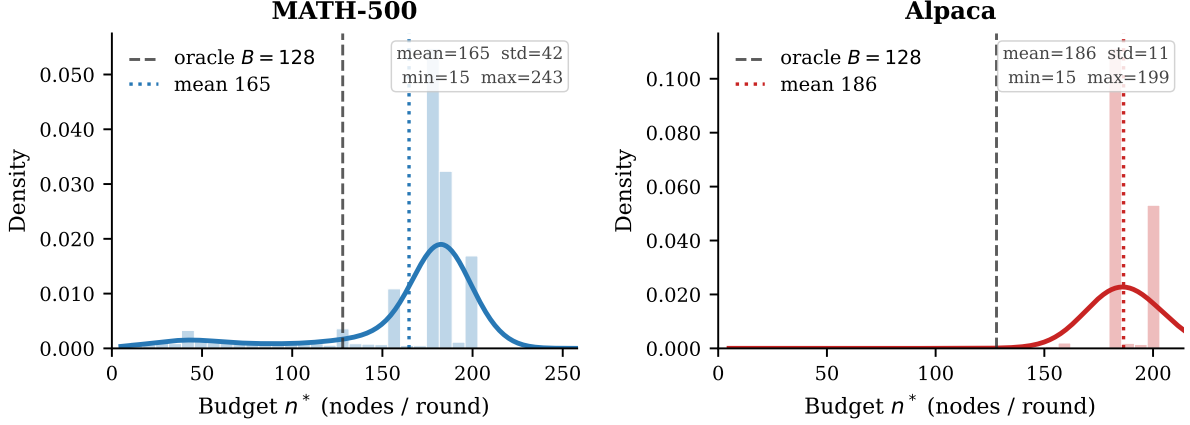


Figure 4: Per-round budget n^* distribution for CADDTREE on MATH-500 and Alpaca (Qwen3-8B, temp. 0.0). On MATH-500, the budget varies widely (std ≈ 42), spanning both below and above the grid oracle ($B = 128$). On Alpaca, the distribution is tightly concentrated (std ≈ 11) near 186, consistently above the oracle. In both cases CADDTREE selects n^* automatically each round without any manual tuning.

Hardware and hyperparameters. All experiments run on 8 A800 GPUs with block size $L = 16$, bfloat16 precision, maximum 2048 new tokens, and temperatures $\{0.0, 1.0\}$. For CADDTREE, cost parameters are estimated once per (model, hardware) pair via the profiling procedure in Section 4.6. For DDTree and CADDTREE, the target model uses standard PyTorch scaled dot product attention, as the standard FlashAttention-2 (Dao, 2024) kernel does not support the custom tree attention mask; the DFlash drafter uses FlashAttention-2. For the AR and DFlash baselines, we report the faster of the two attention implementations.

5.2 Main Results

Table 1 reports per-token generation time (ms/tok; lower corresponds to higher throughput) and mean acceptance length τ at temperature 0.0. Temperature 1.0 results are in Appendix C.

CADDTREE matches or surpasses DDTree-oracle on nearly all tasks. For Qwen3-8B, the grid oracle is constrained to $B=128$; CADDTREE selects budgets between grid values and outperforms the oracle on most tasks. For Qwen3-4B, the oracle uses $B=256$; CADDTREE closely matches the oracle across tasks without any grid search. The pattern is consistent at both temperatures (temperature-1.0 results in Appendix C).

5.3 Cost Model Validation

Figure 3 shows measured verification latency against node count for nine context lengths (0–8k tokens). The measured latency is consistent with

Table 2: Cost model ablation (Qwen3-4B, temp. 0.0). Per-token latency in ms/tok; lower is better. “Constant” sets $C_v(n; \ell) = C_v(0; \ell)$ for all n , causing the algorithm to always expand to the maximum budget; latency increases about $2\times$ across tasks.

C_v model	MATH-500	AIME 2025	Alpaca
CADDTREE	4.53	4.96	10.66
Constant	9.14	10.25	19.37

being monotone and convex in node count across all contexts; the convex regression fits closely track the measurements ($R^2 \geq 0.979$ across all nine context lengths, RMSE ≤ 4.9 ms), supporting Assumption 1. Moreover, both the base verification cost $C_v(0; \ell)$ and the marginal cost per node increase with context length; by design, this causes CADDTREE to select smaller budgets at longer contexts without any manual adjustment. Table 2 compares two cost model variants.

The constant model sets $C_v(n; \ell) = C_v(0; \ell)$ for all n , so the surrogate throughput is strictly increasing and the algorithm always grows the tree to the maximum budget (capped at 1024 nodes), yielding a clear per-token latency increase and confirming that per-node verification overhead is non-negligible.

5.4 Adaptive Budget Analysis

The throughput-optimal n^* varies substantially across rounds, even within the same dataset. Figure 4 makes this concrete: on MATH-500, budgets spread widely, with a substantial fraction of rounds selecting fewer nodes than the grid oracle and a

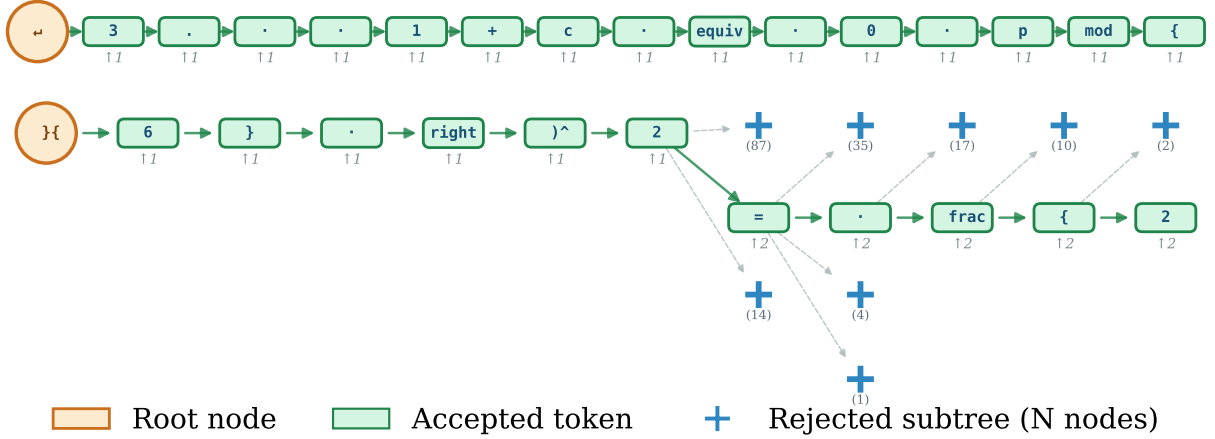


Figure 5: Throughput-optimal draft trees constructed by CADDTREE for two decoding rounds (MATH-500, Qwen3-8B, temp. 0.0), showing the rank of each accepted token among its sibling candidates. The orange circle is the bonus token (tree root); green boxes are accepted tokens labeled with their rank ($\uparrow k$: the accepted token was the k -th most probable child); a + marker with count N below it indicates a rejected subtree of N nodes. *Top* ($n^* = 15$, 15 tokens accepted): all accepted tokens are rank-1 with no competing siblings; the drafter is near-certain and a narrow chain suffices, saving 87% of the oracle budget. *Bottom* ($n^* = 181$, 13 tokens accepted): accepted tokens span multiple ranks and large rejected subtrees appear at every step; wide exploration is necessary to find the accepted path. The two regimes coexist within the same dataset; no fixed budget serves both, and CADDTREE selects n^* automatically each round.

larger fraction selecting more. No single fixed budget serves both regimes well. On Alpaca, the distribution is tightly concentrated and the mean falls between grid points, at a value no fixed grid budget matches exactly. CADDTREE finds the per-round optimum automatically in both cases.

Figure 5 reveals *why* the optimal budget differs: the drafter’s per-token uncertainty varies across steps. When the drafter is confident, every accepted token sits at rank 1 with no competing siblings; a narrow chain suffices, saving most of the oracle budget at no loss in accepted tokens. When the drafter is uncertain, accepted tokens span multiple ranks and large rejected subtrees appear at every step; the tree must fan out widely to find the accepted path. Together, the two panels confirm that per-round budget adaptation captures both regimes, and that no single fixed budget can eliminate this variation.

5.5 Ablation: Sensitivity to Cost Parameters

Table 3 shows that all four perturbations produce $|\Delta| \leq 0.07$ ms/tok, suggesting that approximate cost estimates suffice and highly precise offline calibration is not strictly necessary.

Table 3: Sensitivity of CADDTREE per-token latency to cost parameter perturbations (Qwen3-4B, MATH-500, temp. 0.0). Δ is the change relative to the nominal (unperturbed) run.

	$C_d \times 0.5$	$C_d \times 2.0$	$C_v \times 0.5$	$C_v \times 2.0$
Δ (ms/tok)	-0.07	+0.05	-0.04	-0.01

6 Conclusion

We introduced CADDTREE, a cost-aware diffusion draft tree method that optimizes per-token latency directly rather than acceptance length under a fixed budget. By explicitly modeling both drafting and verification latencies, we showed that the throughput objective decomposes into a one-dimensional budget search that is efficiently solvable via a greedy stopping rule with provable unimodality guarantees under convex verification cost. CADDTREE eliminates offline budget selection and adapts the tree size per decoding round, consistently matching or exceeding DDTREE with oracle budget selection across diverse benchmarks and target models.

References

- Zihao An, Huajun Bai, Ziqiong Liu, Dong Li, and Emad Barsoum. 2025. Pard: Accelerating llm inference with low-cost parallel draft model adaptation. *arXiv preprint arXiv:2504.18583*.
- Marianne Arriola, Aaron Gokaslan, Justin Chiu, Zhihan Yang, Zhixuan Qi, Jiaqi Han, Subham Sahoo, and Volodymyr Kuleshov. 2025. Block diffusion: Interpolating between autoregressive and diffusion language models. In *International Conference on Learning Representations*, volume 2025, pages 50726–50753.
- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and 1 others. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*.
- Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D Lee, Deming Chen, and Tri Dao. 2024. Medusa: Simple llm inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*.
- Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. 2023. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*.
- Jian Chen, Yesheng Liang, and Zhijian Liu. 2026. Dflash: Block diffusion for flash speculative decoding. *arXiv preprint arXiv:2602.06036*.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, and 1 others. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, and 1 others. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.
- Tri Dao. 2024. Flashattention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations*, volume 2024, pages 35549–35562.
- Paul J Goulart and Yuwen Chen. 2026. Clarabel: An interior-point solver for conic programs with quadratic objectives. *Mathematical Programming Computation*, pages 1–83.
- Naman Jain, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2025. Livecodebench: Holistic and contamination free evaluation of large language models for code. In *International Conference on Learning Representations*, volume 2025, pages 58791–58831.
- Wonseok Jeon, Mukul Gagrani, Raghavv Goel, Junyoung Park, Mingyu Lee, and Christopher Lott. 2024. Recursive speculative decoding: Accelerating llm inference via sampling without replacement. *arXiv preprint arXiv:2402.14160*.
- Tao Jin, Phuong Minh Nguyen, and Naoya Inoue. 2026. Goose: Anisotropic speculation trees for training-free speculative decoding. *arXiv preprint arXiv:2604.02047*.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, pages 19274–19286. PMLR.
- Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2024a. Eagle-2: Faster inference of language models with dynamic draft trees. In *Proceedings of the 2024 conference on empirical methods in natural language processing*, pages 7421–7432.
- Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2024b. Eagle: Speculative sampling requires rethinking feature uncertainty. *arXiv preprint arXiv:2401.15077*.
- Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2026. Eagle-3: Scaling up inference acceleration of large language models via training-time test. *Advances in Neural Information Processing Systems*, 38:136737–136756.
- Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2024. Let’s verify step by step. In *International Conference on Learning Representations*, volume 2024, pages 39578–39601.
- Fuliang Liu, Xue Li, Ketai Zhao, Yinxi Gao, Ziyang Zhou, Zhonghui Zhang, Zhibin Wang, Wanchun Dou, Sheng Zhong, and Chen Tian. 2026a. Dart: Diffusion-inspired speculative decoding for fast llm inference. *arXiv preprint arXiv:2601.19278*.
- Tianyu Liu, Qitan Lv, Yuhao Shen, Xiao Sun, and Xiaoyan Sun. 2026b. Talon: Confidence-aware speculative decoding with adaptive token trees. *arXiv preprint arXiv:2601.07353*.
- Xupeng Miao, Gabriele Oliaro, Zihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, and 1 others. 2024. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, pages 932–949.
- Liran Ringel and Yaniv Romano. 2026. Accelerating speculative decoding with block diffusion draft trees. *arXiv preprint arXiv:2604.12989*.

- Benjamin Spector and Chris Re. 2023. Accelerating llm inference with staged speculative decoding. *arXiv preprint arXiv:2308.04623*.
- Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B Hashimoto. 2023. Alpaca: A strong, replicable instruction-following model. *Stanford Center for Research on Foundation Models*. <https://crfm.stanford.edu/2023/03/13/alpaca.html>, 3(6):7.
- Jikai Wang, Yi Su, Juntao Li, Qingrong Xia, Zi Ye, Xinyu Duan, Zhefeng Wang, and Min Zhang. 2025. Opt-tree: Speculative decoding with adaptive draft tree structure. *Transactions of the Association for Computational Linguistics*, 13:188–199.
- Yunfan Xiong, Ruoyu Zhang, Yanzeng Li, and Lei Zou. 2025. Dyspec: Faster speculative decoding with dynamic token tree structure. *World Wide Web*, 28(3):36.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, and 1 others. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.
- Jiebin Zhang, Zhenghan Yu, Liang Wang, Nan Yang, Eugene J Yu, Zheng Li, Yifan Song, Dawei Zhu, Xingxing Zhang, Furu Wei, and 1 others. 2026. Learning to draft: Adaptive speculative decoding with reinforcement learning. *arXiv preprint arXiv:2603.01639*.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, and 1 others. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems*, 36:46595–46623.

Appendix

A Proofs

A.1 Derivation of Per-Round Objective

Over R rounds, the response throughput is $\Theta_R = \sum_{t=1}^R (1 + \alpha_t) / \sum_{t=1}^R C(\mathcal{T}_t; \ell_t)$. At each round, $\mathcal{T}_t$ is a deterministic function of the drafter's marginals $\{q_i^{(t)}\}$, while acceptance α_t is random under the target p . By the law of large numbers, as $R \rightarrow \infty$:

$$\Theta_R \rightarrow \Theta = \frac{\mathbb{E}_t[\mathbb{E}_p[1 + \alpha_t \mid \mathcal{T}_t]]}{\mathbb{E}_t[C(\mathcal{T}_t; \ell_t)]} \approx \frac{\mathbb{E}_t[1 + \Phi(\mathcal{T}_t)]}{\mathbb{E}_t[C(\mathcal{T}_t; \ell_t)]},$$

where the inner expectation over target acceptance is replaced by the surrogate Φ . We therefore maximize $\hat{\theta}(\mathcal{T}_t; \ell_t)$ at each round as a tractable per-round surrogate for the long-run ratio Θ . $\square$

A.2 Proof of Proposition 1

Fixed-budget optimality. For any node $\mathbf{s} = (s_1, \dots, s_d)$, its parent satisfies $\pi_{\mathbf{s}_-} = \pi_{\mathbf{s}}/q_d(s_d) \geq \pi_{\mathbf{s}}$, so every ancestor has weakly higher prefix probability. Hence if $\mathbf{s}$ is among the top- n nodes by π , every ancestor is too: $\mathcal{T}_n^*$ is prefix-closed and forms a valid tree. Since $\Phi(\mathcal{T}) = \sum_{\mathbf{s} \in \mathcal{T}} \pi_{\mathbf{s}}$ and $\mathcal{T}_n^*$ selects the n nodes with the largest $\pi_{\mathbf{s}}$, no valid n -node tree achieves a higher Φ .

Budget selection. For fixed n , the denominator $C_d + C_v(n; \ell)$ is constant, so maximizing $\hat{\theta}(\mathcal{T}; \ell)$ over valid n -node trees reduces to maximizing $\Phi(\mathcal{T})$, achieved by $\mathcal{T}_n^*$. Taking the maximum over $n \geq 0$ yields (8). $\square$

A.3 Proof of Lemma 2

We verify the weighted average representation by direct expansion. Let $c_{n+1} = C_v(n+1; \ell) - C_v(n; \ell) > 0$ and $w_n = (C_d + C_v(n; \ell)) / (C_d + C_v(n+1; \ell))$, so $1 - w_n = c_{n+1} / (C_d + C_v(n+1; \ell))$. Then:

$$\begin{aligned} w_n \hat{\theta}^*(n) + (1 - w_n) \rho_{n+1} &= \frac{1 + \Phi^*(n)}{C_d + C_v(n+1; \ell)} + \frac{c_{n+1}}{C_d + C_v(n+1; \ell)} \cdot \frac{\delta_{n+1}}{c_{n+1}} \\ &= \frac{1 + \Phi^*(n) + \delta_{n+1}}{C_d + C_v(n+1; \ell)} = \hat{\theta}^*(n+1). \end{aligned}$$

The equivalence $\hat{\theta}^*(n+1) > \hat{\theta}^*(n)$ iff $\rho_{n+1} > \hat{\theta}^*(n)$ follows immediately: $\hat{\theta}^*(n+1)$ is a strict convex combination of $\hat{\theta}^*(n)$ and ρ_{n+1} , so it lies between them, and exceeds $\hat{\theta}^*(n)$ iff $\rho_{n+1} > \hat{\theta}^*(n)$. $\square$

A.4 Proof of Theorem 3 (Unimodality)

By Assumption 1, $\{\rho_n\}$ is non-increasing. If $\hat{\theta}^*$ is non-decreasing for all n , the conclusion holds trivially. Otherwise, let $n_0 \geq 0$ be the smallest index with $\hat{\theta}^*(n_0 + 1) \leq \hat{\theta}^*(n_0)$, equivalently $\rho_{n_0+1} \leq \hat{\theta}^*(n_0)$. From the weighted average representation:

$$\begin{aligned} \hat{\theta}^*(n_0 + 1) &= w_{n_0} \hat{\theta}^*(n_0) + (1 - w_{n_0}) \rho_{n_0+1} \\ &\geq \rho_{n_0+1}. \end{aligned} \quad (14)$$

Inductive step. Assume $\rho_{n+1} \leq \hat{\theta}^*(n)$ for some $n \geq n_0$, so $\hat{\theta}^*(n+1) \leq \hat{\theta}^*(n)$ and the bound above gives $\hat{\theta}^*(n+1) \geq \rho_{n+1}$. Since $\{\rho_n\}$ is non-increasing:

$$\rho_{n+2} \leq \rho_{n+1} \leq \hat{\theta}^*(n+1),$$

so $\hat{\theta}^*(n+2) \leq \hat{\theta}^*(n+1)$ by Eq. (11). The induction is complete: $\hat{\theta}^*$ is non-increasing for all $n \geq n_0$. $\square$

A.5 Proof of Corollary 4 (Greedy Optimality)

Algorithm 1 terminates at the first n for which $\hat{\theta}^*(n+1) \leq \hat{\theta}^*(n)$. By Theorem 3, this is the global maximizer n^* . At termination, the heap has popped exactly the top- n^* prefixes by probability (by Proposition 1), and Proposition 1 guarantees that $\mathcal{T}_{n^*}$ is the throughput-optimal tree. $\square$

A.6 Proof of Corollary 5 (Convex Cost)

$\{\delta_n\}$ is non-increasing by construction; $\{c_n\}$ non-decreasing by the convexity hypothesis; hence $\rho_n = \delta_n / c_n$ is non-increasing. $\square$

B Benchmark Details

All runs use block size 16, bfloat16 precision, and a maximum of 2048 new tokens. Each dataset uses 128 evaluated examples, except AIME 2025 (30), HumanEval (164), and MT-Bench (160). The evaluation set is sharded across 8 A800 GPUs. Before timing, one warmup prompt is run through each method (autoregressive, DFlash, DDTree, CAD-DTREE) to exclude one-time setup costs.

C Temperature 1.0 Results

Table 4: Per-token generation time (ms/tok) and mean acceptance length τ at temperature 1.0. n^* is the oracle grid budget (same as T=0 grid search); $\bar{n}^*(\sigma)$ is CADDTREE’s per-round mean (std). CADDTREE achieves the lowest per-token latency without offline budget search.

Dataset	AR	DFlash		DDTree-oracle			CADDTREE (ours)		
	ms/tok	ms/tok	τ	ms/tok	τ	n^*	ms/tok	τ	$\bar{n}^*(\sigma)$
<i>Qwen3-4B</i>									
MATH-500	30.76	6.95	6.68	5.17	9.04	256	5.20	9.26	224 (51)
GSM8K	31.05	7.64	6.00	5.50	8.65	256	5.56	8.67	250 (59)
AIME 2025	30.60	9.17	5.04	6.34	7.27	128	6.58	7.39	220 (31)
HumanEval	31.01	7.61	5.94	5.44	8.73	256	5.49	8.64	227 (55)
MBPP	30.79	8.08	5.66	5.64	8.43	256	5.80	8.32	260 (57)
LiveCodeBench	30.70	7.06	6.61	5.36	8.75	128	5.35	9.05	215 (38)
MT-Bench	31.09	13.53	4.01	9.02	5.84	256	8.94	6.05	235 (36)
Alpaca	30.66	16.71	2.98	10.96	4.63	256	11.20	4.63	261 (40)
<i>Qwen3-8B</i>									
MATH-500	31.10	7.31	6.46	5.48	8.73	128	5.40	9.02	168 (36)
GSM8K	30.84	7.69	5.92	5.67	8.27	128	5.62	8.44	184 (23)
AIME 2025	31.05	9.65	4.94	6.98	6.94	128	6.80	7.11	158 (33)
HumanEval	31.73	8.33	5.54	5.93	7.99	128	5.96	8.11	174 (34)
MBPP	31.60	8.95	5.21	6.27	7.62	128	6.18	7.89	187 (18)
LiveCodeBench	30.83	6.96	6.84	5.30	9.02	128	5.29	9.24	162 (33)
MT-Bench	30.92	14.46	3.78	9.68	5.54	128	9.47	5.77	169 (28)
Alpaca	31.67	17.47	2.94	11.64	4.44	128	11.55	4.54	186 (11)